

Moving Average Process

Course: Practical Time Series Analysis

Lecturer: Dr. David Zmiaikou

April 9, 2024

Contents

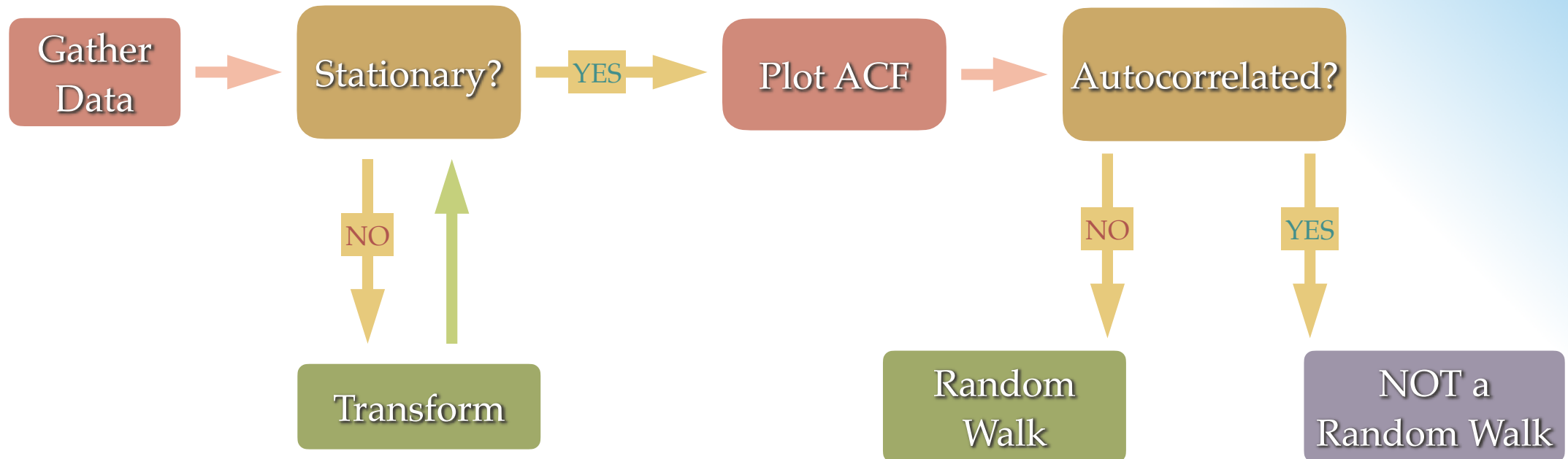
01 Forecasting a Random Walk

02 Modelling a Moving Average Process

03 Identifying the Order of a Moving Average Process

04 Implementation

Identifying a Random Walk



In the context of time series, **a random walk** is defined as a series whose first difference is *stationary* and *uncorrelated*.

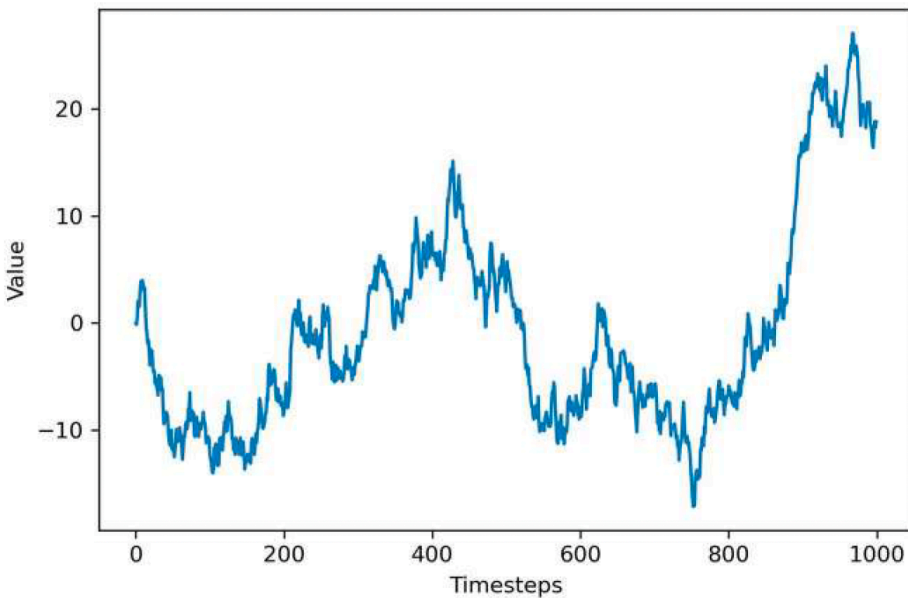
Forecasting with Statistical Models

- When performing statistical modelling, we need to perform hypothesis testing, study our data carefully to extract its properties, and find the best model for our data.
- We will develop $MA(q)$ models, $AR(p)$ models, $ARMA(p, q)$ models, $ARIMA(p, d, q)$ models for non-stationary time series, $SARIMA(p, d, q)(P, D, Q)_m$ for seasonal time series, and SARIMAX models to include external variables in your forecast.

01

Forecasting a Random Walk

def What is a Random Walk Process?

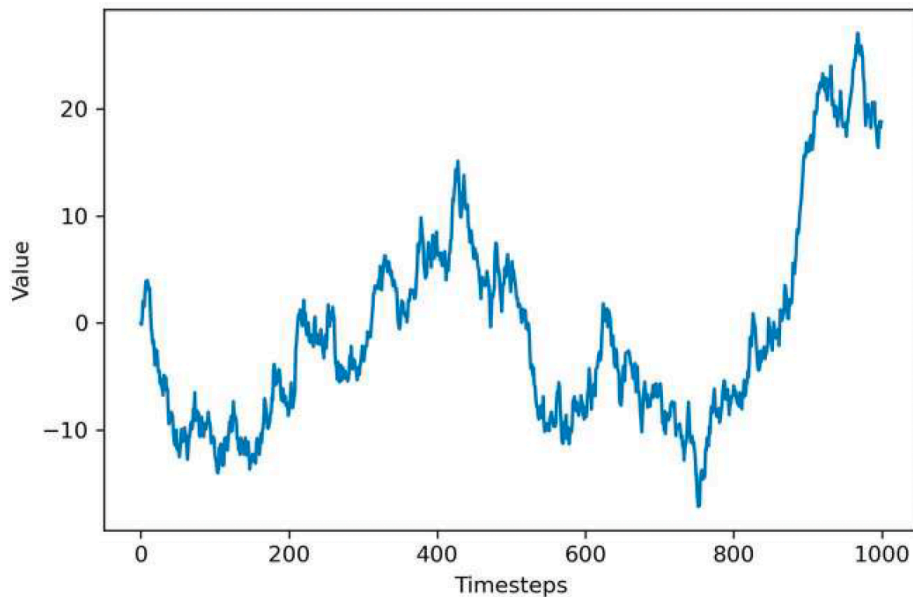


- Mathematically speaking, in a random walk process, the present value y_t is the sum of the previous timestep y_{t-1} , a constant C and a random number ϵ_t :

$$y_t = C + y_{t-1} + \epsilon_t.$$

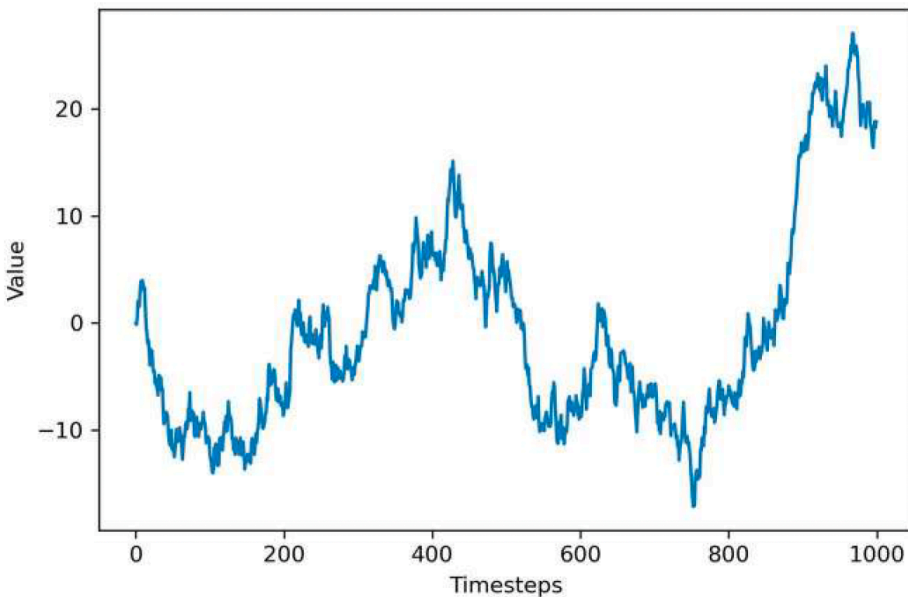
The random variables $\epsilon_1, \epsilon_2, \dots$ are independent and normally distributed with mean 0 and variance 1. That is why they are also called *white noise*.

Forecasting a Random Walk Process



- Forecasting a random walk might sound surprising, since we established that a random walk takes random steps as time progresses.
- Since the values change randomly, no statistical learning model can be applied.
- Predicting a random change is impossible, unless we predict a random value ourselves, which is not ideal.
- We can only use naive forecasting methods, or baselines.
- Depending on the use case, your forecasting horizon will vary.

Forecasting on a Long Horizon

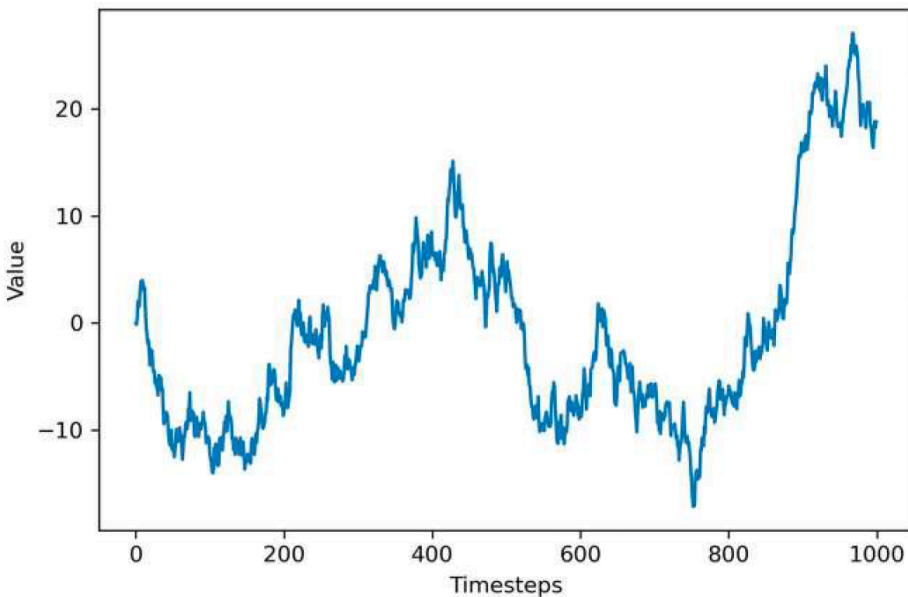


- This is not an ideal case — a random walk can unexpectedly increase or decrease because past observations are not predictive of changes in the future.
- The train set will contain the first 800 timesteps, which corresponds to 80% of the simulated data:

```
import pandas as pd  
df = pd.DataFrame({'value': random_walk})
```

```
train = df[:800]  
test = df[800:]
```

Forecasting on a Long Horizon: Mean

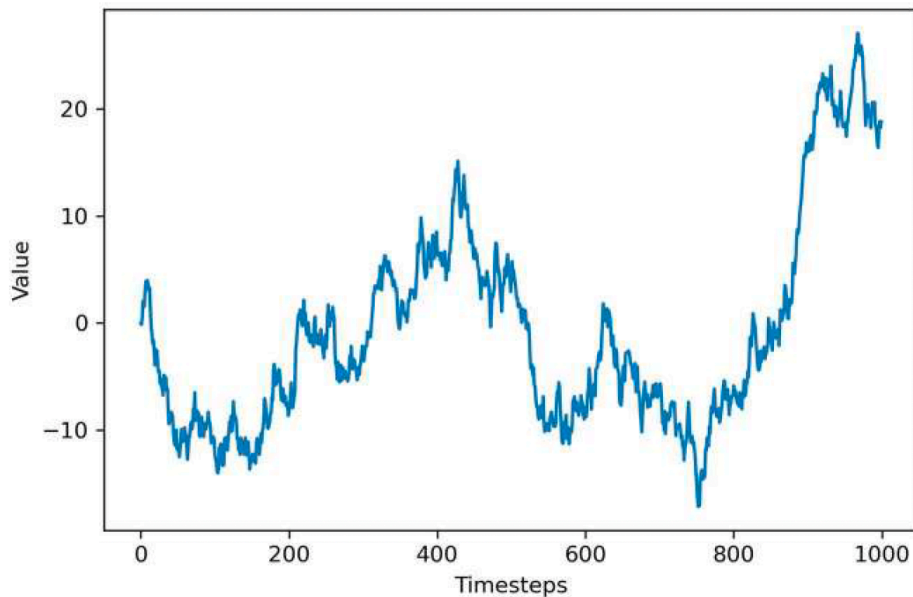


- Forecasting the mean is fairly straightforward. We'll simply calculate the mean of the train set and say that the next 200 timesteps will be equal to that value:

```
mean = np.mean(train.value)  
test.loc[:, 'pred_mean'] = mean
```

We get a historical mean of -3.68 . This means that we'll forecast that the next 200 timesteps of our simulated random walk will have a value of -3.68 .

Forecasting on a Long Horizon: Last Known Value

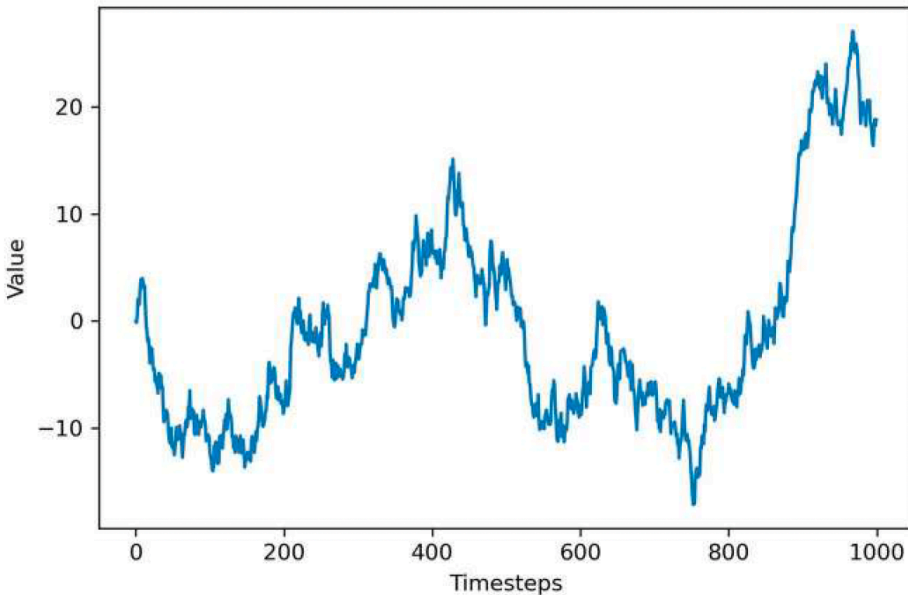


- Another possible baseline is to predict the last known value of the train set. Here, we'll simply extract the last value of the train set and assign its value as our prediction for the next 200 timesteps:

```
last_value = train.iloc[-1].value  
test.loc[:, 'pred_last'] = last_value
```

This method yields forecasts with a constant value of **-6.81**.

Forecasting on a Long Horizon: Drift



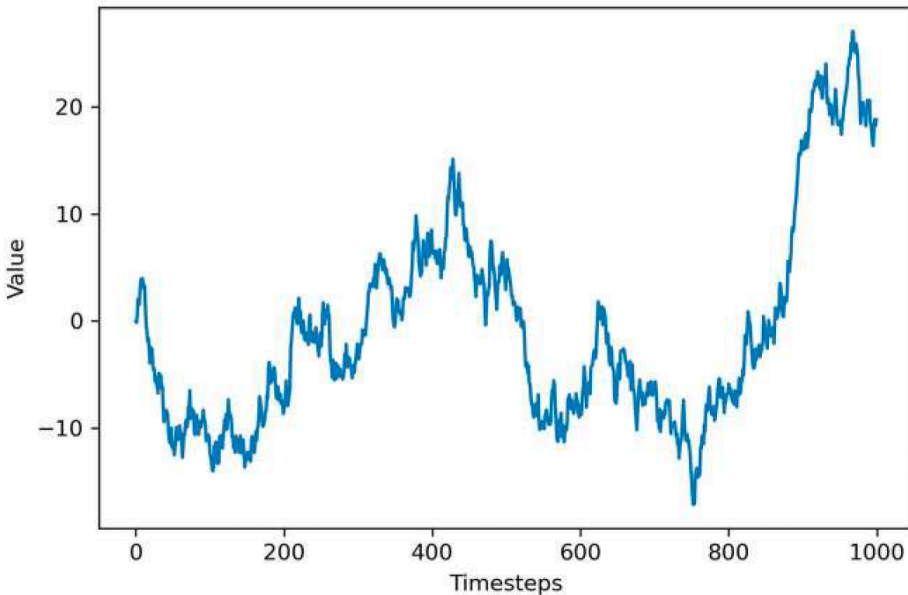
- The drift method is a modification of predicting the last known value.
- We allow the values to increase or decrease over time. The rate at which values will change in the future is equal to that seen in the train set.
- It is equivalent to calculating the slope between the first and last value of the train set and simply extrapolating this straight line into the future:

$$\text{slope} = \frac{\Delta y}{\Delta x} = \frac{y_{\text{last}} - y_{\text{first}}}{\text{\#timesteps} - 1}$$

- The drift method simply states that the value of our forecast is linearly dependent on the timestep:

$$\text{forecast} = \text{slope} \times \text{timestep} + y_{\text{first}}$$

Forecasting on a Long Horizon: Drift



- Implementation in Python:

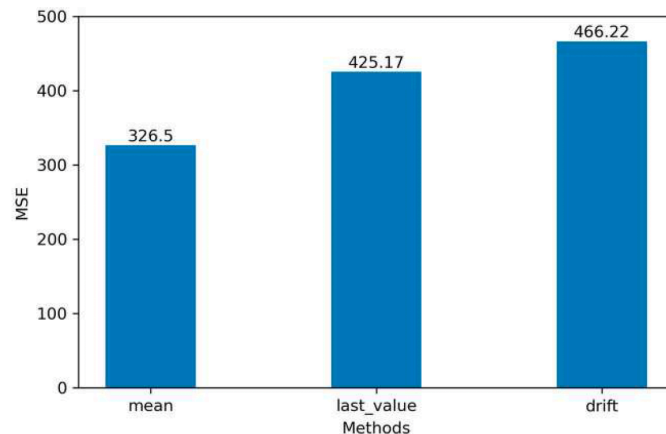
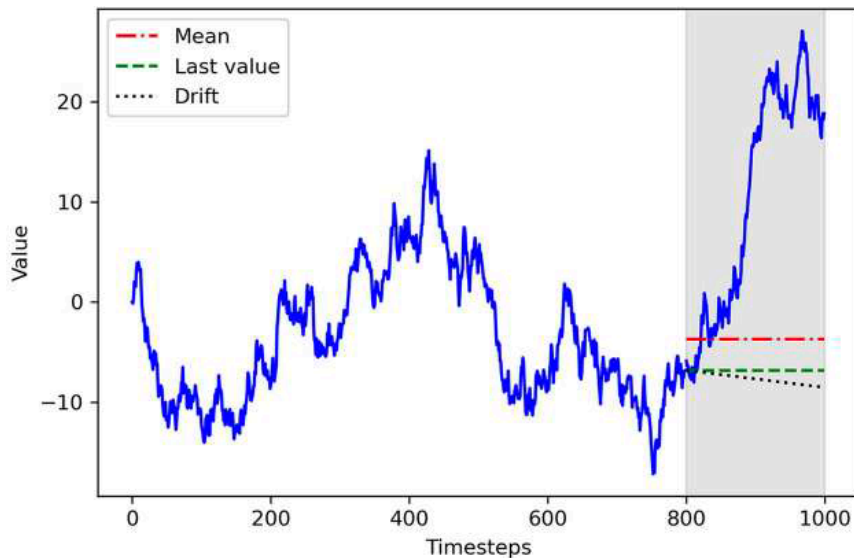
```
deltaX = 800 - 1  
deltaY = last_value - 0  
drift = deltaY / deltaX
```

This gives us a drift of -0.0085 , which means that the values of our forecasts will slowly decrease over time.

- Multiply each timestep by the drift to get our forecast values:

```
x_vals = np.arange(800, 1001, 1)  
pred_drift = drift * x_vals  
test.loc[:, 'pred_drift'] = pred_drift
```

Forecasting on a Long Horizon (Comparison)



- Measure the MSE:

```
from sklearn.metrics import mean_squared_error
```

```
mse_mean = mean_squared_error(test['value'],  
test['pred_mean'])
```

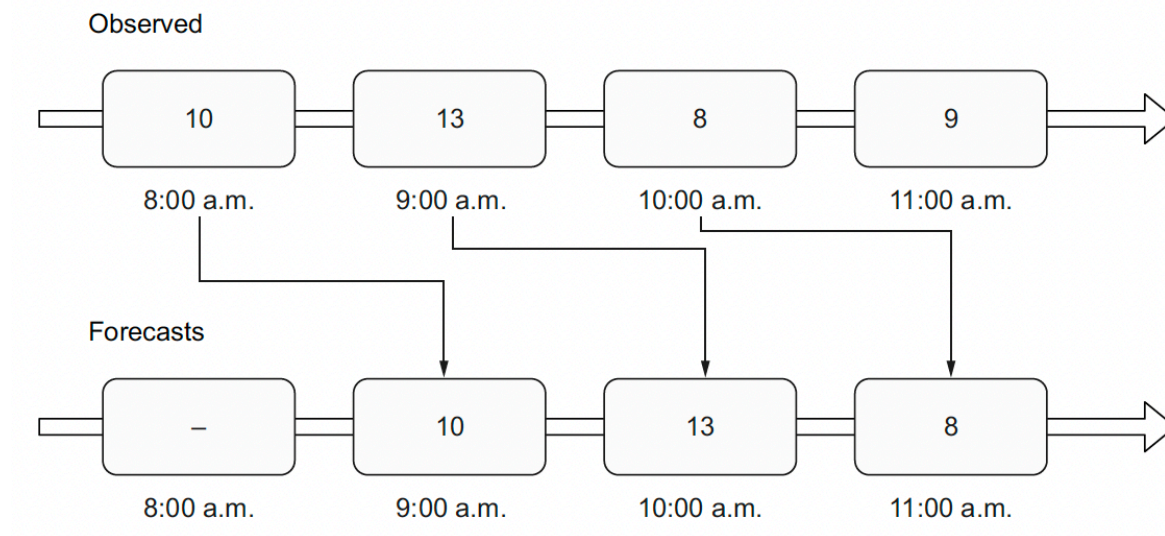
```
mse_last = mean_squared_error(test['value'], test['pred_last'])
```

```
mse_drift = mean_squared_error(test['value'], test['pred_drift'])
```

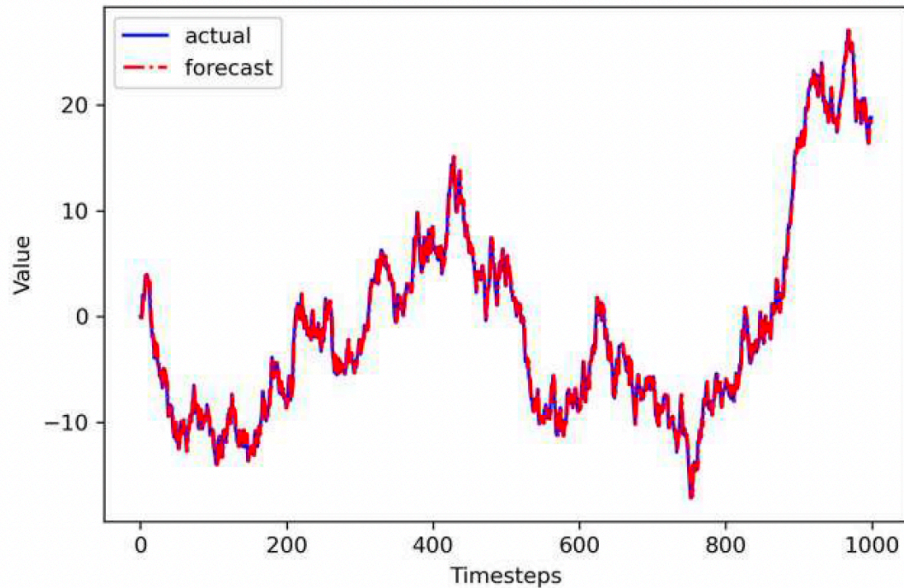
- The best forecast was obtained by predicting the historical mean, and yet the MSE exceeds 300. This is an extremely high value considering that our simulated random walk does not exceed the value of 30.
- By now, you should be convinced that forecasting a random walk on a long horizon does not make sense.

Forecasting the Next Timestep

- The method: we take an observed value and use it to predict the next timestep.



Forecasting the Next Timestep



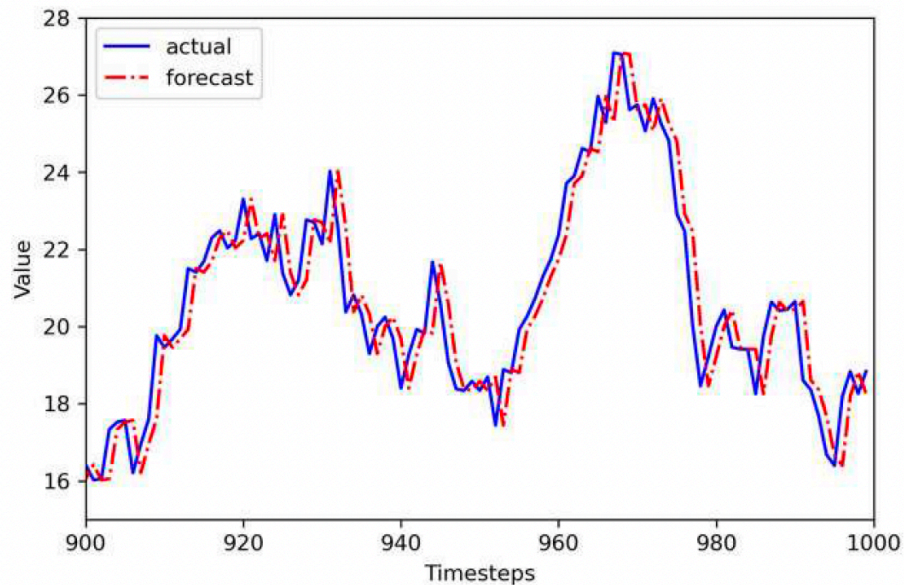
- The **pandas** library has a **shift** method:

`df_shift = df.shift(periods=1)`

*`mse_one_step = mean_squared_error(test['value'],
df_shift[800:])`*

- This yields a value of 0.93, which again might lead us to think that we have a very performant model, since the MSE is very close to 0. However, we know that we are simply forecasting the value observed at the previous timestep.

Forecasting the Next Timestep



- However, we know that we are simply forecasting the value observed at the previous timestep.
- Therefore, if a random walk process must be forecast, it is better to make many short-term forecasts.
- That way, we do not allow for many random numbers to accumulate over time, which will degrade the quality of our forecasts in the long term.

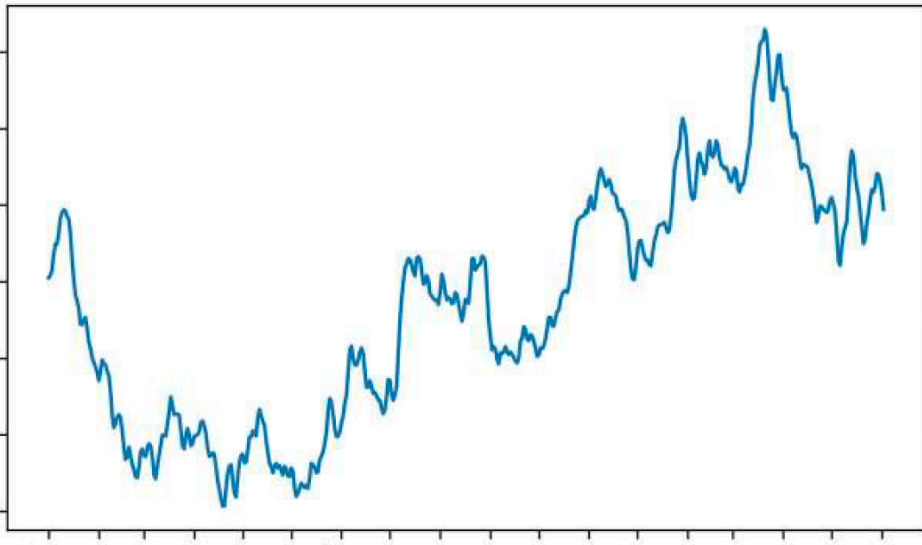
What is next?

- We defined a random walk process as a series whose first difference is stationary with no autocorrelation. This means that plotting its ACF will show no significant coefficients after lag 0.
- However, it is possible that a stationary process may still exhibit autocorrelation. In this case, we have a time series that can be approximated by
 - a moving average model $MA(q)$,
 - an autoregressive model $AR(p)$,
 - or an autoregressive moving average model $ARMA(p, q)$.

02

Modelling a Moving Average Process

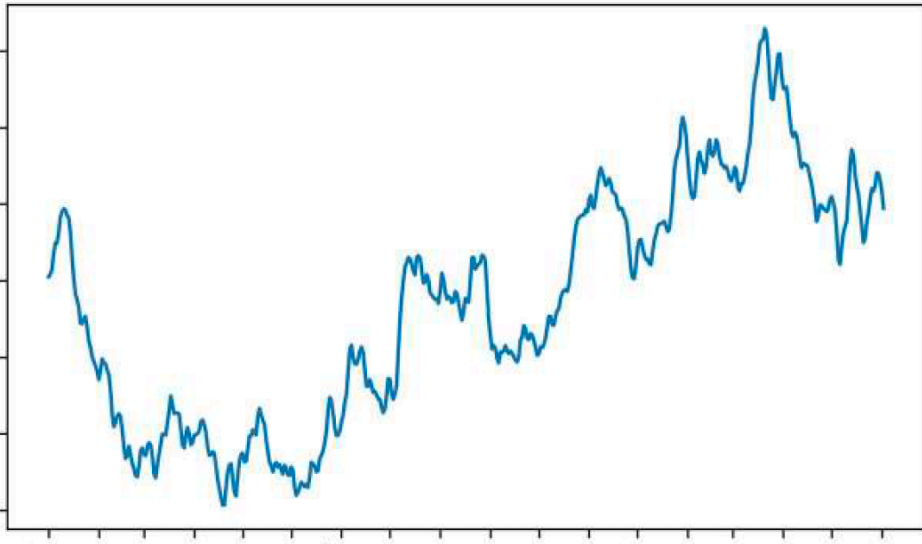
^{def} What is a Moving Average Process?



- In a **moving average (MA) process**, the current value depends linearly on the mean of the series, the current error term, and past error terms.
- The moving average model is denoted as $MA(q)$, where q is the **order**. The general expression of an $MA(q)$ model is

$$y_t = \mu + \epsilon_t + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \cdots + \theta_q \epsilon_{t-q}.$$

$\stackrel{\text{def}}{=}$ What is a Moving Average Process?

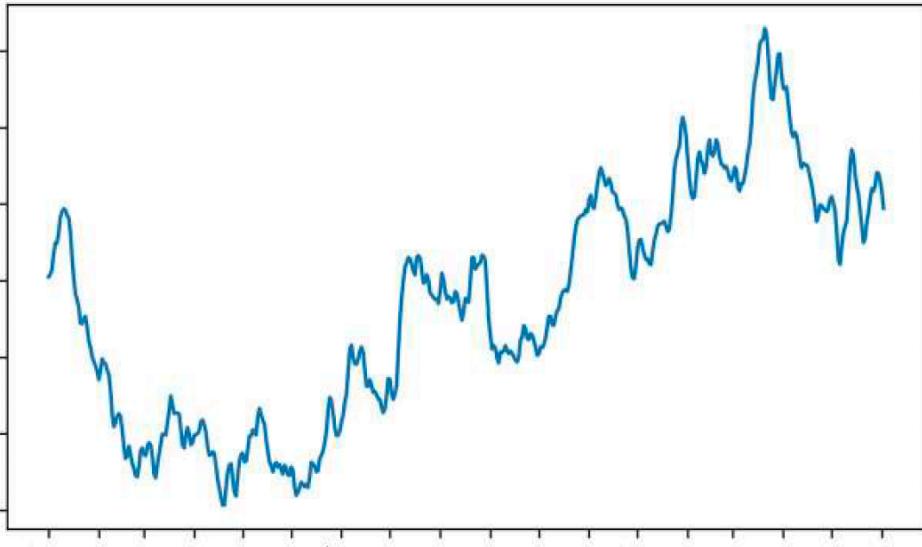


- In a **moving average (MA) process**, the current value depends linearly on the mean of the series, the current error term, and past error terms.
- The moving average model is denoted as $MA(q)$, where q is the **order**. The general expression of an $MA(q)$ model is

$$y_t = \mu + \epsilon_t + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \cdots + \theta_q \epsilon_{t-q}.$$

- The error terms are assumed to be mutually *independent* and *normally distributed*, just like white noise.
- The magnitude of the impact of past errors on the present value is quantified using a coefficient.

$\stackrel{\text{def}}{=}$ What is a Moving Average Process?



- The **order** q of the $MA(q)$ process affects the number of past error terms that must be included in the model. The larger q is, the more past error terms affect the present value.
- Therefore, it is important to determine the order of the moving average process in order to fit the appropriate model.

03

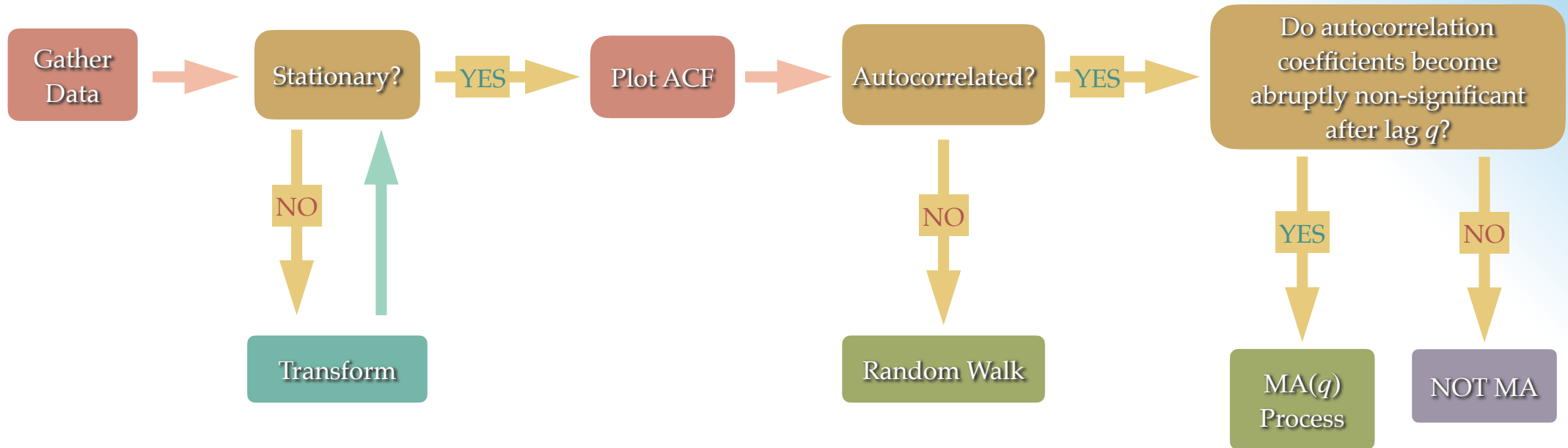
Identifying the Order of a Moving Average Process



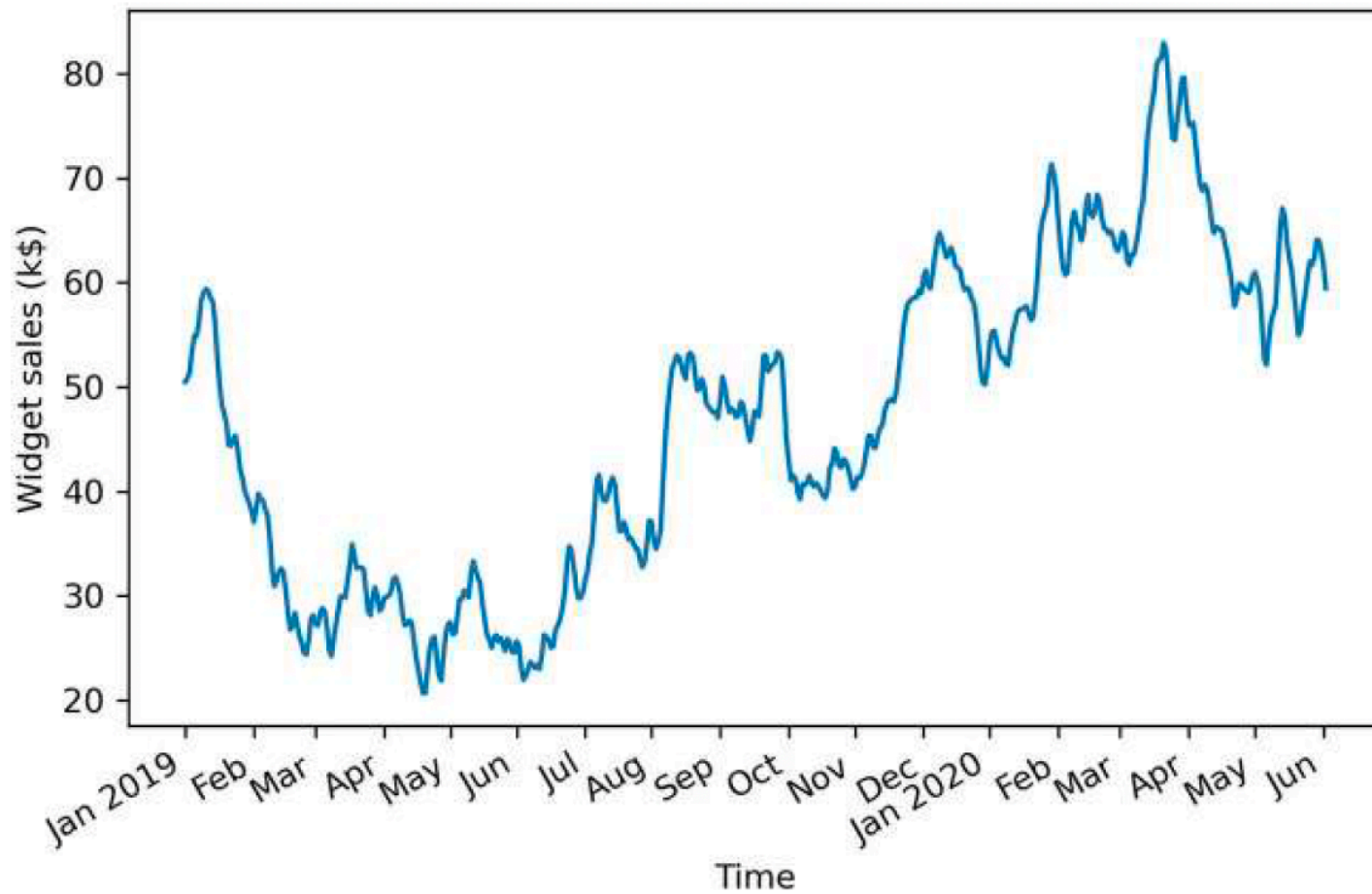
EPIC

Institute of Technology
Powered by ePam

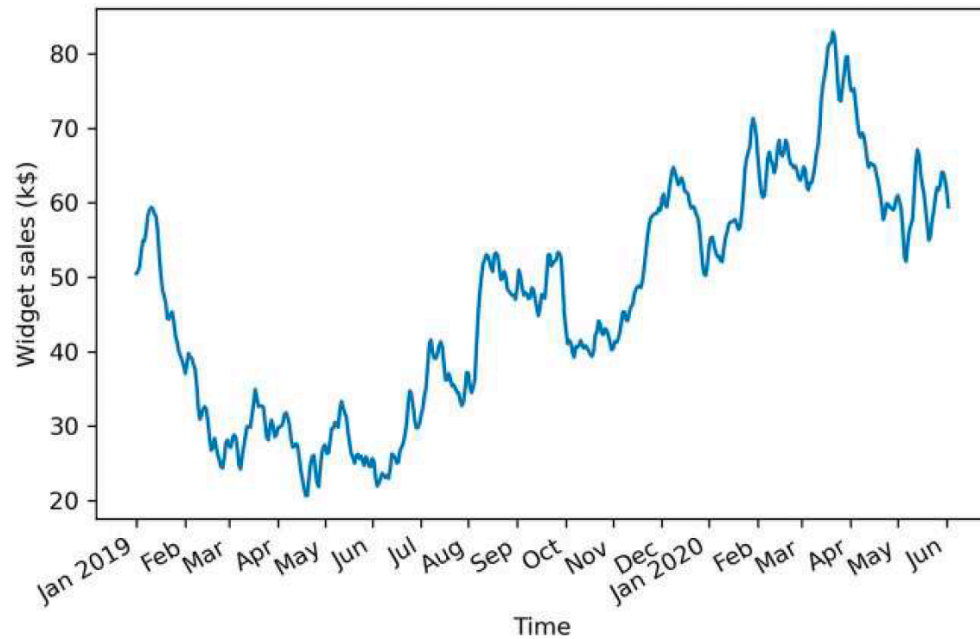
Identifying the Order of a Moving Average Process



Example 1: Widget sales for the XYZ Widget Company over 500 days



Example 1: Widget sales for the XYZ Widget Company over 500 days



- There is a long-term trend.
- Not stationary.
- No apparent cyclical pattern.

04

Implementation

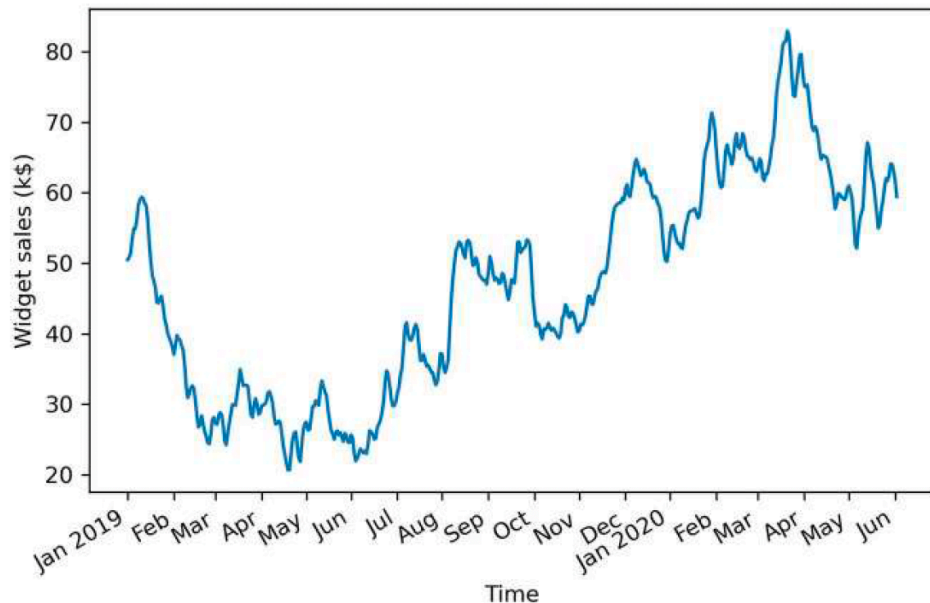
Implementation

- We will use **DataFrame** from **pandas**:

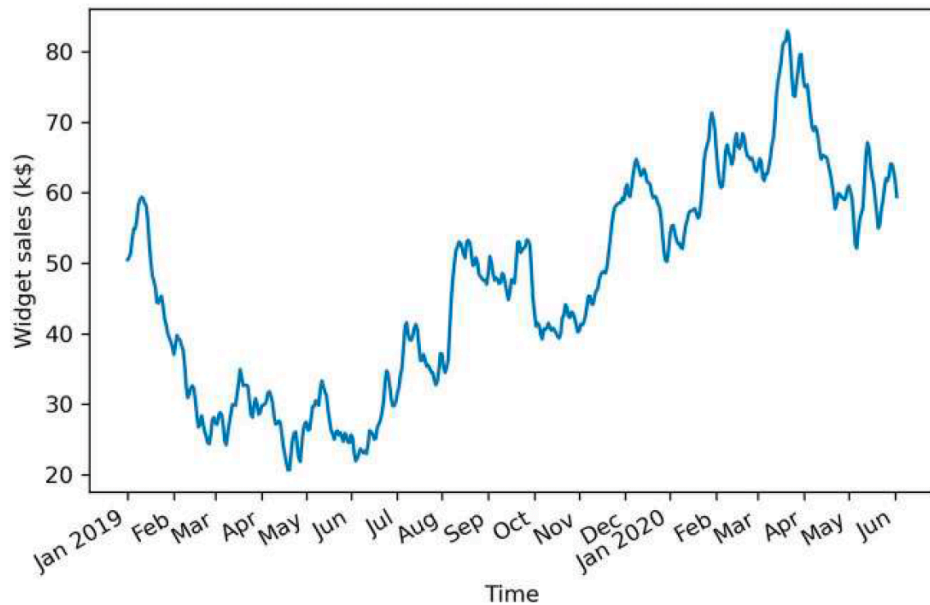
```
import pandas as pd  
df = pd.read_csv('widget_sales.csv')
```

- Use the **adfuller** function from the **statsmodels** library and extract the ADF statistic and p-value. If the ADF statistic is a large negative number and the p-value is smaller than 0.05, our series is stationary. Otherwise, we must apply transformations.

```
from statsmodels.tsa.stattools import adfuller  
ADF_result = adfuller(df['widget_sales'])  
print(f'ADF Statistic: {ADF_result[0]}')  
print(f'p-value: {ADF_result[1]}')
```



Implementation



- We will use **DataFrame** from **pandas**:

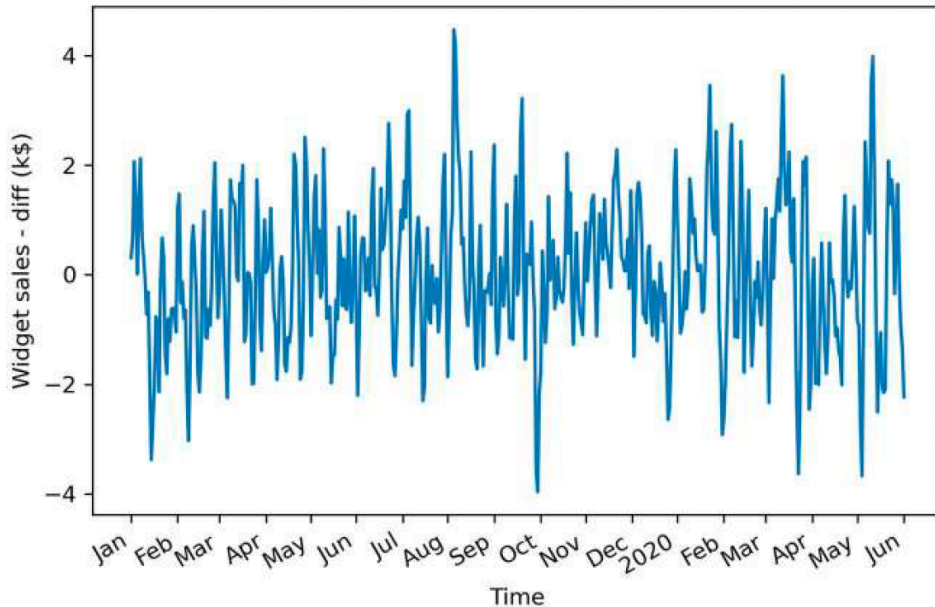
```
import pandas as pd  
df = pd.read_csv('widget_sales.csv')
```

- Use the **adfuller** function from the **statsmodels** library and extract the ADF statistic and p-value.

```
from statsmodels.tsa.stattools import adfuller  
ADF_result = adfuller(df['widget_sales'])  
print(f'ADF Statistic: {ADF_result[0]}')  
print(f'p-value: {ADF_result[1]}')
```

- This results in an ADF statistic of -1.51 and a p-value of 0.53 . Not stationary.
- Try to de-trend by applying a first-order differencing. Use the **diff** method from the **numpy** library:

```
import numpy as np  
widget_sales_diff = np.diff(df['widget_sales'], n=1)
```



Implementation

- Try to de-trend by applying a first-order differencing. Use the **diff** method from the **numpy** library:

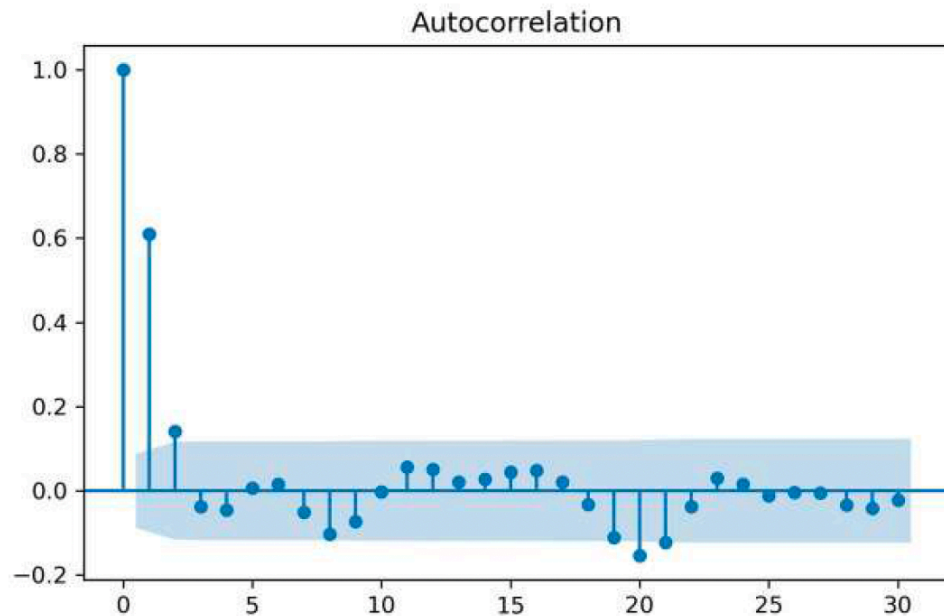
```
import numpy as np  
widget_sales_diff = np.diff(df['widget_sales'], n=1)
```

- Again check *stationarity*:

```
ADF_result = adfuller(df['widget_sales_diff'])  
print(f'ADF Statistic: {ADF_result[0]}')  
print(f'p-value: {ADF_result[1]}')
```

This gives an ADF statistic of -10.6 and a p-value very small. So, the new series is stationary.

Implementation



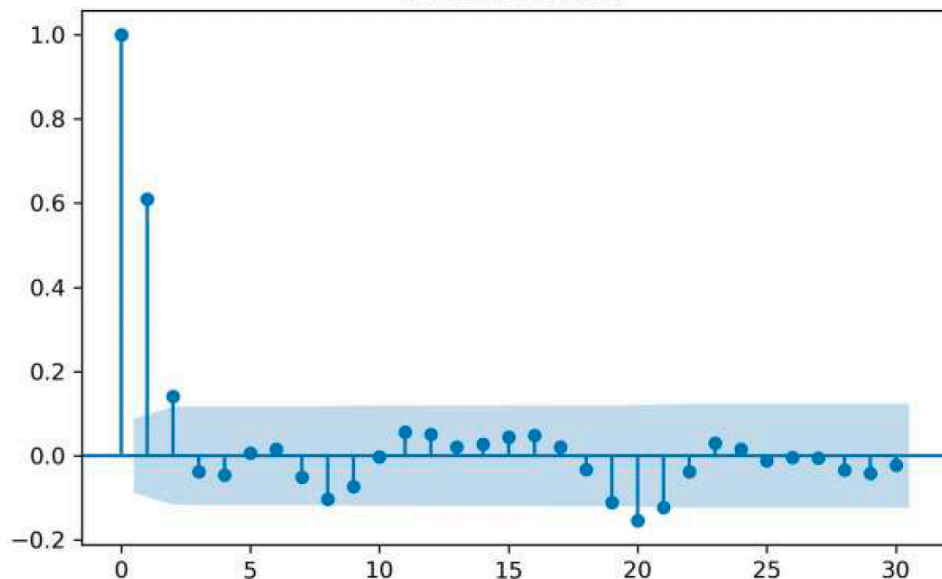
- The next step is to check whether the series is *autocorrelated*. Use the **plot_acf** function from the **statsmodels** library:

```
from statsmodels.graphics.tsaplots import plot_acf
```

```
plot_acf(widget_sales_diff, lags=30)
```

Implementation

Autocorrelation



- The next step is to check whether the series is *autocorrelated*. Use the **plot_acf** function from the **statsmodels** library:

```
from statsmodels.graphics.tsaplots import plot_acf
```

```
plot_acf(widget_sales_diff, lags=30)
```

- There are significant coefficients up **until lag 2**. Then they abruptly become non-significant, as they remain in the shaded area of the plot.
- This means that we have a **stationary moving average process of order 2**. We can use a second-order moving average model, or **MA(2) model**, to forecast our stationary time series.



EPIC

Institute of Technology
Powered by <epam>

Thank you!